

Projeto e Análise de Algoritmos

Recorrências

Thiago Pinheiro de Araújo

EMAp/FGV

2025.2

Apresentar **recorrências** como solução para avaliar a complexidade de algoritmos recursivos e técnicas para resolvê-las.

Definição e exemplos

- Uma **recorrência** é uma equação ou desigualdade que descreve uma função em termos do seu próprio valor em entradas menores
 - É útil para descrever o comportamento de algoritmos recursivos.
- **Exemplo 1: Fatorial**

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 0 \\ T(n - 1) + \theta(1) & \text{se } n > 0 \end{cases}$$

■ Exemplo 2: Fibonacci

```
def fibonacci(n):  
    if n == 0 or n == 1:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

$$T(n) = \begin{cases} \theta(1) & \text{se } n \leq 1 \\ T(n-1) + T(n-2) + \theta(1) & \text{se } n > 1 \end{cases}$$

■ Exemplo 3: Busca binária

```
def binary_search(values, low, high, value):  
    if high >= low:  
        mid = (high + low) // 2  
        if values[mid] == value:  
            return mid  
        elif values[mid] > value:  
            return binary_search(values, low, mid - 1, value)  
        else:  
            return binary_search(values, mid + 1, high, value)  
    else:  
        return -1
```

$$T(n) \leq \begin{cases} \theta(1) & \text{se } n = 0 \\ T(n/2) + \theta(1) & \text{se } n > 0 \end{cases}$$

- Ótimo, uma relação de recorrência consegue descrever o comportamento de um algoritmo recursivo
 - Porém, como comparar com outros algoritmos?
 - Precisamos resolver a recorrência!
- Vamos apresentar 4 métodos:
 - Método da substituição.
 - Método da árvore de recursão.
 - Método da iteração.
 - Método mestre.

Método da substituição

- O **método da substituição** consiste em provar por indução que uma determinada função $f(n)$ pressuposta é a solução para a recorrência.
- Esse método somente é aplicável quando é possível gerar uma hipótese sobre a solução.
- É necessário provar exatamente a hipótese
 - Não podem ser aplicadas simplificações utilizando a notação assintótica.
- O método pode ser utilizado para provar tanto limites superiores como inferiores de uma recorrência.

■ Exemplo 1:

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

- Vamos supor que a solução seja $T(n) = O(n^2)$
- Precisamos então provar que $T(n) \leq c n^2$
- Caso base: $T(1) = 1 \Rightarrow c \cdot 1^2 = c$, logo $c \geq 1$ **ok!**
- Avaliamos a suposição para $n/2$: $T\left(\frac{n}{2}\right) \leq c \left(\frac{n}{2}\right)^2 = \frac{cn^2}{4}$
- Realizamos a substituição: $T(n) \leq \frac{2cn^2}{4} + n \leq cn^2 \Rightarrow n \leq \frac{cn^2}{2} \Rightarrow c \geq \frac{2}{n}$
- Escolhendo $c = 2$ temos que $c \geq 2/n$ será válido para todo $n \geq 1$
- Portanto $T(n)$ é $O(n^2)$

■ Exemplo 2:

- Vamos supor que a solução seja $T(n) = O(n)$
- Precisamos então provar que $T(n) \leq c n$
- Caso base: $T(1) = 1 \Rightarrow c \cdot 1 = c$, logo $c \geq 1$ **ok!**
- Avaliamos a suposição para $n/2$: $T\left(\frac{n}{2}\right) \leq \frac{cn}{2}$
- Realizamos a substituição:

$$T(n) \leq \frac{2cn}{2} + 1 \leq cn$$
$$cn + 1 \leq cn \text{ **impossível!**}$$

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + 1 & \text{se } n > 1 \end{cases}$$

■ Exemplo 2:

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + 1 & \text{se } n > 1 \end{cases}$$

- Podemos modificar nossa hipótese inicial para $T(n) \leq c n - d$

- Caso base: $T(1) = 1 \Rightarrow c \cdot 1 - d = c - d \geq 1$

- Avaliamos a suposição para $n/2$: $T\left(\frac{n}{2}\right) \leq \frac{cn}{2} - d$

- Realizamos a substituição:

$$T(n) \leq 2\left(\frac{cn}{2} - d\right) + 1 \leq cn - d$$

$$cn - 2d + 1 \leq cn - d$$

$$d \geq 1$$

- Logo podemos escolher $d = 1$ e $c = 2$
- Portanto $T(n)$ é $O(n)$

- **Exercício:**

- Prove que $T(n)$ é $O(n \log(n))$
- Precisamos então provar que $T(n) \leq cn \log(n)$
- Avaliamos a suposição para $n/2$: $T\left(\frac{n}{2}\right) \leq c \frac{n}{2} \log\left(\frac{n}{2}\right)$
- Realizamos a substituição

$$\begin{aligned} T(n) &\leq 2c \frac{n}{2} \log\left(\frac{n}{2}\right) + n \\ &\leq cn \log(n) - cn + n \end{aligned}$$

- Avaliamos que

$$\begin{aligned} cn \log(n) - cn + n &\leq cn \log(n) \\ cn &\geq n \\ c &\geq 1 \text{ ok!} \end{aligned}$$

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

- Avaliamos o caso base:
 $T(1) = 1$
 $1 \leq c \cdot 1 \cdot \log(1)$
 $1 \leq 0$ não pode!
- Precisamos encontrar um n_0 que satisfaça a hipótese:
 $T(2) = 4$
 $4 \leq c \cdot 2 \cdot \log(2)$
 $4 \leq 2c$
 $c \geq 2$ ok!
- Portanto $T(n)$ é $O(n \log(n))$

Método da árvore de recursão

- O **método da árvore de recursão** consiste em construir uma árvore definindo em cada nível os sub-problemas gerados pela iteração do nível anterior
 - A forma geral é encontrada ao somar o custo de todos os nós.
- Características da árvore:
 - Cada nó representa um subproblema.
 - Os filhos de cada nó representam as suas chamadas recursivas.
 - O valor do nó representa o custo computacional do respectivo problema.
- Esse método é útil para analisar algoritmos de divisão e conquista.

Método da árvore de recursão

■ **Exemplo:**
$$T(n) \leq \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{se } n > 1 \end{cases}$$

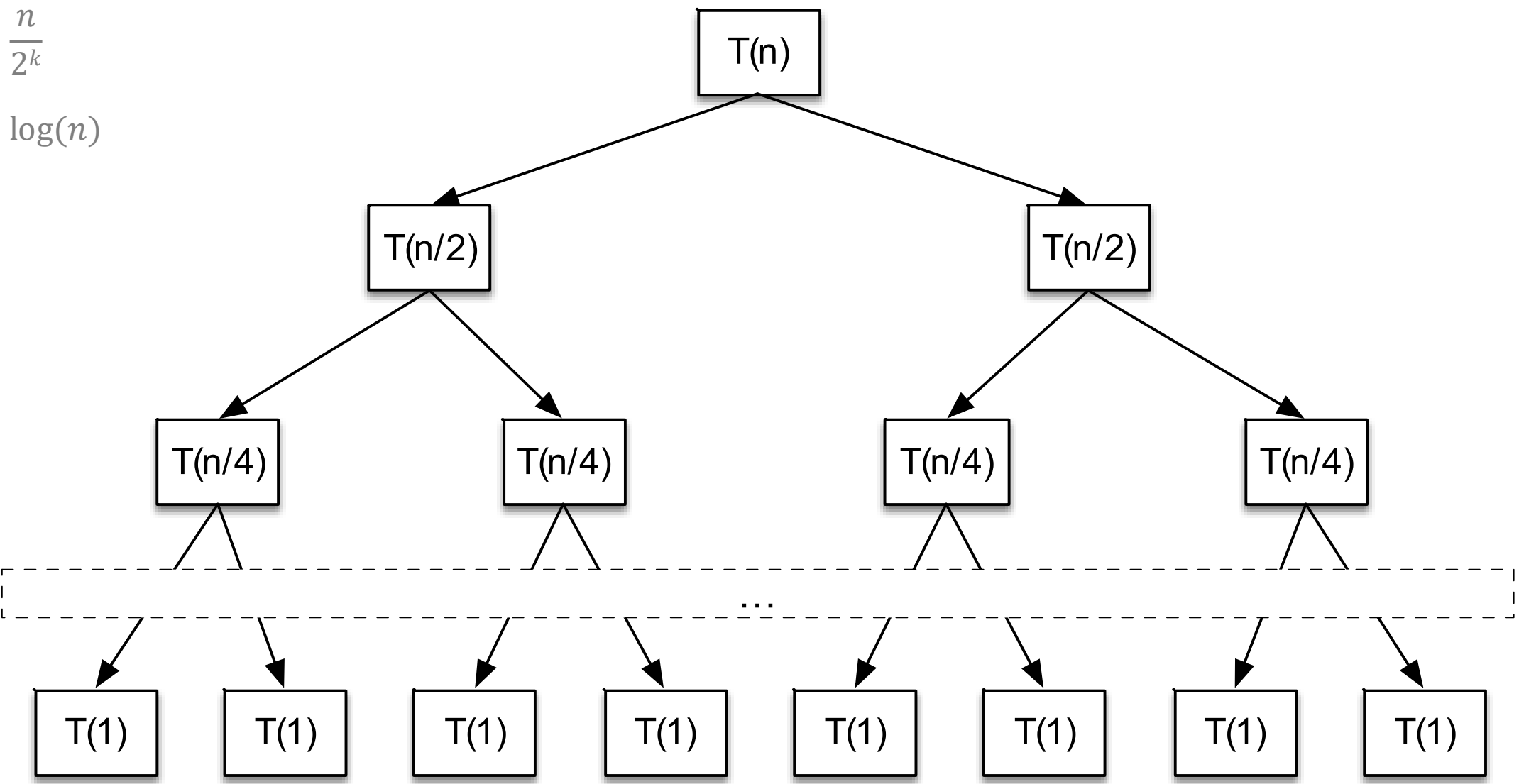
$$T(n) = \sum_{k=0}^{\log(n)} 2^k \left(\frac{n}{2^k}\right) = n \log(n) + n$$

Portanto $T(n)$ é $O(n \log(n))$

Calculando a altura:

$$1 = \frac{n}{2^k}$$

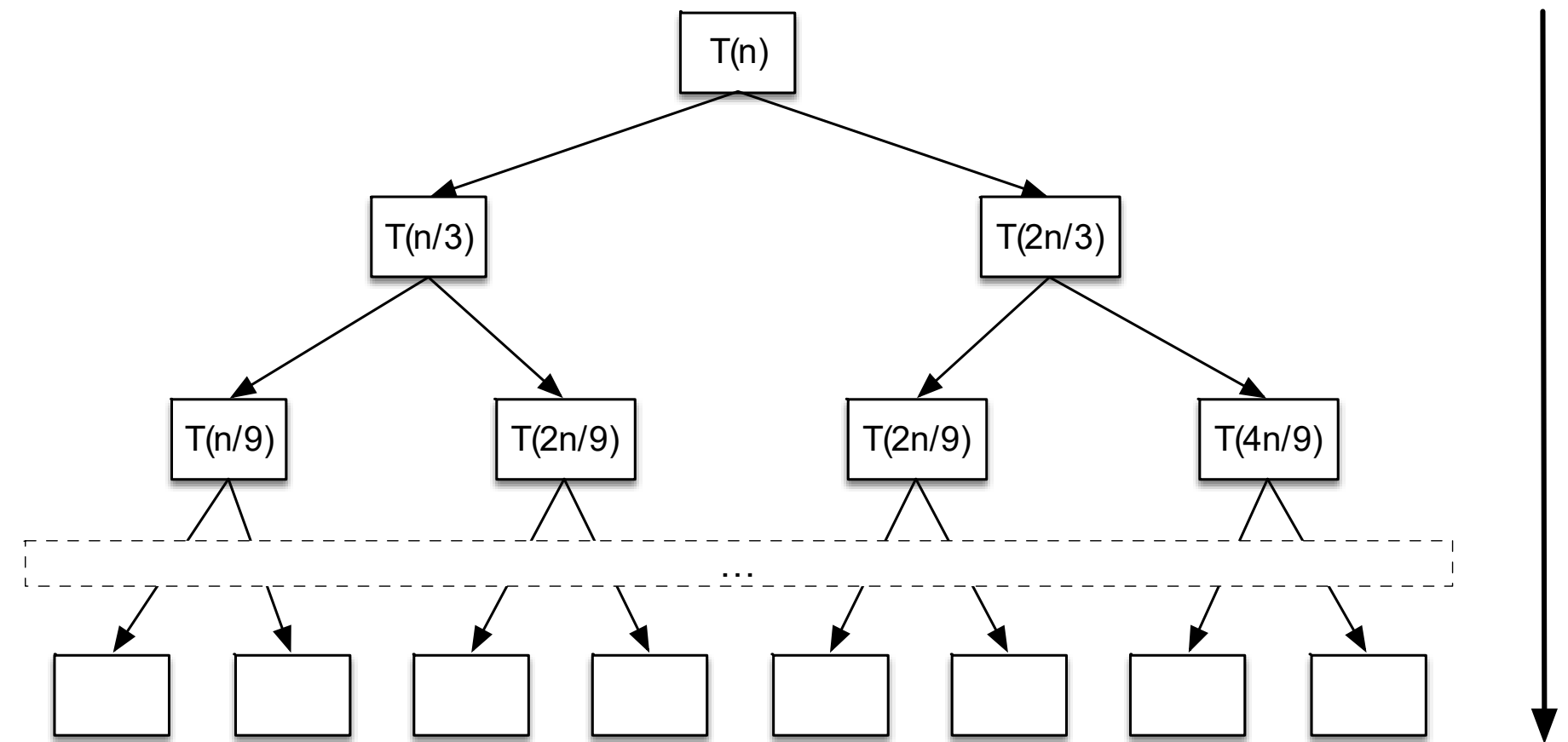
$$k = \log(n)$$



iteração	# nós	custo por nó
0	1	n
1	2	$\frac{n}{2}$
2	4	$\frac{n}{4}$
k	2^k	$\frac{n}{2^k}$
$\log(n)$	n	1

■ Exercício:

$$T(n) \leq \begin{cases} \theta(1) & \text{se } n = 1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n & \text{se } n > 1 \end{cases}$$



- Observe que a árvore é desbalanceada.
- Cada nível computa n elementos.
- O caminho mais longo apresenta o padrão: $(2/3)^k n$ para o nível k .
- A altura da árvore pode ser calculada por:
$$(2/3)^k n = 1 \Rightarrow n = (3/2)^k \Rightarrow k = \log_{3/2}(n)$$
- Portanto $T(n)$ é $O(n \log_{3/2}(n)) = O(n \log(n))$

Método da iteração

- O **método da iteração** consiste em expandir a relação de recorrência até o n-ésimo termo, de forma que seja possível compreender a sua forma geral.

- **Exemplo 1:**

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 0 \\ T(n-1) + 1 & \text{se } n > 0 \end{cases}$$

$$\begin{aligned} T(0) &= 1 \\ n - k &= 0 \\ k &= n \end{aligned}$$

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= T(n-2) + 1 + 1 \\ &= T(n-3) + 1 + 1 + 1 \\ &\quad \dots \\ &= T(n-k) + k \end{aligned}$$

$$\begin{aligned} T(n) &= T(n-k) + k \\ \text{se } k &= n \\ T(n) &= T(0) + n \end{aligned}$$

Portanto $T(n)$ é $\theta(n)$

■ Exemplo 2:

$$T(n) \leq \begin{cases} \theta(1) & \text{se } n = 1 \\ T\left(\frac{n}{2}\right) + 1 & \text{se } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{2}\right) + 1 \\ &\leq T\left(\frac{n}{4}\right) + 1 + 1 \\ &\leq T\left(\frac{n}{8}\right) + 1 + 1 + 1 \\ &\leq T\left(\frac{n}{16}\right) + 1 + 1 + 1 + 1 \\ &\quad \vdots \\ &\leq T\left(\frac{n}{2^k}\right) + k \end{aligned}$$

Buscando obter $T(1) = 1$:

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log(n)$$

Logo podemos escrever:

$$T(n) \leq T(1) + \log(n)$$

Portanto $T(n)$ é $O(\log(n))$

■ Exemplo 3:

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T(n-1) + n & \text{se } n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n-1) + n \\ &= 2(2T(n-1-1) + n-1) + n = 4T(n-2) + 3n-2 \\ &= 4(2T(n-2-1) + n-2) + 3n-2 = 8T(n-3) + 7n-10 \\ &= 8(2T(n-3-1) + n-3) + 7n-10 = 16T(n-4) + 15n-34 \\ &\quad \dots \\ &= 2^k T(n-k) + (2^k - 1)n + \sum_{j=1}^{k-1} 2^j (-j) \end{aligned}$$

- **Exemplo 3:**

$$T(n) = 2^k T(n - k) + (2^k - 1)n + \sum_{j=1}^{k-1} 2^j (-j)$$

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T(n - 1) + n & \text{se } n > 1 \end{cases}$$

Para obter $n - k = 1$ temos que $k = n - 1$

$$\begin{aligned} T(n) &= 2^{n-1} T(n - n + 1) + (2^{n-1} - 1)n + \sum_{j=1}^{n-2} 2^j (-j) \\ &= 2^{n-1} T(1) + (2^{n-1} - 1)n + \sum_{j=1}^{n-2} 2^j (-j) \end{aligned}$$

■ Exemplo 3:

$$T(n) = 2^{n-1}T(1) + (2^{n-1}-1)n + \sum_{j=1}^{n-2} 2^j (-j)$$

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T(n-1) + n & \text{se } n > 1 \end{cases}$$

Podemos fazer uma análise mais fraca desconsiderando o somatório:

$$\begin{aligned} T(n) &= 2^{n-1}T(1) + (2^{n-1}-1)n - \sum_{j=1}^{n-2} 2^j j \\ &\leq 2^{n-1}T(1) + (2^{n-1}-1)n \\ &\leq \frac{2^n}{2} + \frac{2^n}{2}n - n \leq \frac{2^n}{2} + \frac{2^n}{2}n \leq \frac{2^n}{2}n + \frac{2^n}{2}n \\ &\leq 2^n n \end{aligned}$$

Portanto $T(n)$ é $O(2^n n)$

■ Exemplo 3:

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T(n-1) + n & \text{se } n > 1 \end{cases}$$

Considerando o somatório temos uma análise mais precisa:

Note que:

$$\sum_{j=1}^{n-2} 2^j j = \frac{1}{2} (2^n n - 3 * 2^n + 4)$$

$$\begin{aligned} T(n) &= 2^{n-1} T(1) + (2^{n-1} - 1)n - \sum_{j=1}^{n-2} 2^j j \\ &= 2^{n-1} T(1) + (2^{n-1} - 1)n - \frac{1}{2} (2^n n - 3 * 2^n + 4) \\ &= 2^n n \left(\frac{1}{2} - \frac{1}{2} \right) + 2^n \left(\frac{1}{2} + \frac{3}{2} \right) - n - 2 \\ &= 2^{n+1} - n - 2 \end{aligned}$$

Portanto $T(n)$ é $\theta(2^n)$

- **Exercício:** resolva a recorrência a seguir

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 0 \\ T(n-1) + n + 1 & \text{se } n > 0 \end{cases}$$

$$\begin{aligned} T(n) &= T(n-1) + n + 1 \\ &= T(n-2) + (n-1) + n + 2 \\ &= T(n-3) + (n-2) + (n-1) + n + 3 \\ &= T(n-4) + (n-3) + (n-2) + (n-1) + n + 4 \\ &\quad \dots \\ &= T(n-k) + (n-k+1) + (n-k+2) + \dots + (n-1) + n + k \end{aligned}$$

- **Exercício:** resolva a recorrência a seguir

$$T(n) = \begin{cases} \theta(1) & \text{se } n = 0 \\ T(n-1) + n + 1 & \text{se } n > 0 \end{cases}$$

$$T(n) = T(n-k) + (n-k+1) + (n-k+2) + \dots + (n-1) + n + k$$

- Para obter $n - k = 0$ temos que $k = n$

$$\begin{aligned} T(n) &= T(n-k) + (n-k+1) + (n-k+2) + \dots + (n-1) + n + k \\ &= T(0) + 1 + 2 + \dots + (n-1) + n + n \\ &= 1 + \frac{n(n+1)}{2} + n = \frac{n^2}{2} + \frac{3n}{2} + 1 \end{aligned}$$

Note que:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Portanto $T(n)$ é $\theta(n^2)$

- O **método mestre** consiste em aplicar o teorema mestre sobre recorrências do tipo

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- Esse teorema pode ser aplicado em algoritmos que dividem a entrada n em a subproblemas de tamanho n/b , com custo por nível definido por $f(n)$.

Método mestre

- O **teorema mestre** é definido por:

- Dada uma recorrência da forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- Considerando $a \geq 1$, $b > 1$ e $f(n)$ assintoticamente positiva.

- Se $f(n) = O(n^{\log b(a) - \varepsilon})$ para alguma constante $\varepsilon > 0$

- Então $T(n) = \Theta(n^{\log b(a)})$

- Se $f(n) = \Theta(n^{\log b(a)})$

- Então $T(n) = \Theta(n^{\log b(a)} \log(n)) = \Theta(f(n) \log(n))$

- Se $f(n) = \Omega(n^{\log b(a) + \varepsilon})$ para alguma constante $\varepsilon > 0$

- Se atender à condição de regularidade

- $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e para todo n suficientemente grande

- Então $T(n) = \Theta(f(n))$

- **Exemplo 1:** $T(n) = 9T\left(\frac{n}{3}\right) + n$
 - $a = 9, b = 3$ e $f(n) = n$
 - $n^{\log_b(a)} = n^{\log_3(9)} = n^{2\log_3(3)} = n^2$
 - Temos que $f(n) = O(n^{\log_b(a) - \varepsilon})$, considerando $\varepsilon = 1$
 - Portanto $T(n)$ é $\theta(n^2)$
- **Exemplo 2:** $T(n) = T\left(\frac{2n}{3}\right) + 1$
 - $a = 1, b = 3/2$ e $f(n) = 1$
 - $n^{\log_b(a)} = n^{\log_{3/2}(1)} = n^0 = 1$
 - Temos que $f(n) = \theta(n^{\log_b(a)})$
 - Portanto $T(n)$ é $\theta(n^{\log_{3/2}(1)} * \log(n)) = \theta(\log(n))$

- **Exemplo 3:** $T(n) = 3T\left(\frac{n}{4}\right) + n\log(n)$
 - $a = 3, b = 4$ e $f(n) = n\log(n)$
 - $n^{\log_b(a)} = n^{\log_4(3)} = n^{0.79}$
 - Temos que $f(n) = \Omega(n^{\log_b(a) + \varepsilon})$, considerando $\varepsilon \sim 0.2$
 - Avaliamos a condição de regularidade:
 - $af\left(\frac{n}{b}\right) \leq cf(n)$
 - $3\left(\frac{n}{4}\log\left(\frac{n}{4}\right)\right) \leq \frac{3}{4}n\log(n)$
 - $\frac{3}{4}n\log(n) \leq cn\log(n) \Rightarrow c \geq 3/4$
 - Portanto $T(n)$ é $\theta(n\log(n))$

- **Exemplo 4:** $T(n) = 2T\left(\frac{n}{2}\right) + n\log(n)$
 - $a = 2, b = 2$ e $f(n) = n\log(n)$
 - $n^{\log_b(a)} = n^{\log_2(2)} = n^1 = n$
 - O caso $f(n) = \Omega(n^{\log_b(a) + \epsilon})$ parece ser a solução
 - Avaliamos a condição de regularidade:
 - $af\left(\frac{n}{b}\right) \leq cf(n)$
 - $2\left(\frac{n}{2}\log\left(\frac{n}{2}\right)\right) \leq cn\log(n)$
 - $n\log(n) - n\log(2) \leq cn\log(n)$
 - $c \geq 1 - \frac{1}{\log(n)}$ **Não será possível determinar um c que também atenda à $c < 1$.**
 - Portanto não é possível aplicar o teorema mestre nessa recorrência.

Método mestre

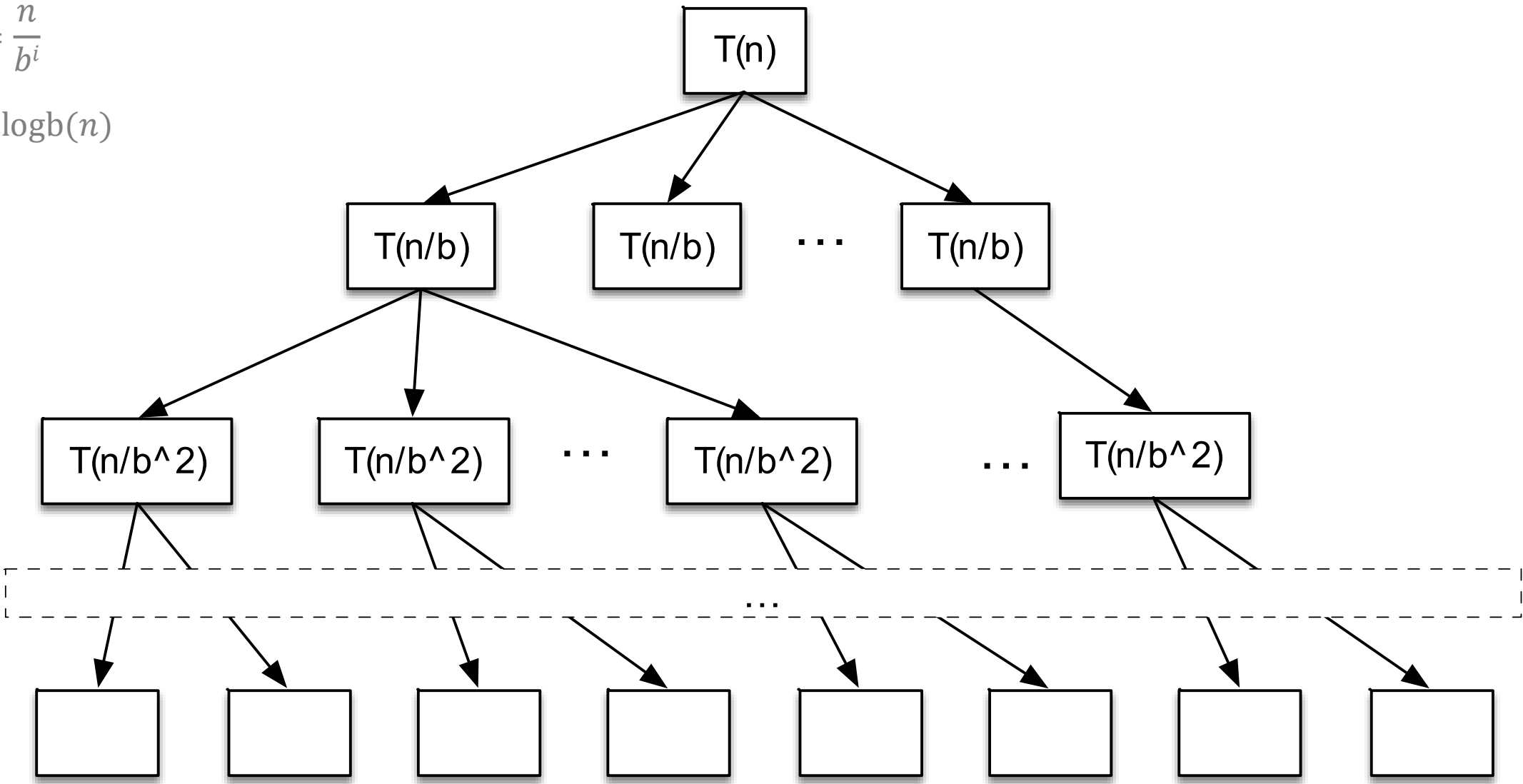
- Considere a forma: $T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k)$

$$T(n) = \sum_{i=0}^{\log(n)} a^i \left(\frac{n}{b^i}\right)^k = \sum_{i=0}^{\log(n)} \frac{a^i}{(b^i)^k} n^k = nk \sum_{i=0}^{\log(n)} \left(\frac{a}{b^k}\right)^i$$

Calculando a altura:

$$1 = \frac{n}{b^i}$$

$$i = \log_b(n)$$



iteração	# nós	tamanho	custo por nó
0	1	n	n^k
1	a	$\frac{n}{b}$	$\left(\frac{n}{b}\right)^k$
2	a^2	$\frac{n}{b^2}$	$\left(\frac{n}{b^2}\right)^k$
i	a^i	$\frac{n}{b^i}$	$\left(\frac{n}{b^i}\right)^k$
$\log_b(n)$	$a^{\log_b(n)}$	1	1^k
$n^{\log_b(a)}$			

- Podemos simplificar o teorema mestre para recorrências da forma:

$$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^k)$$

- Considerando $a \geq 1$, $b > 1$ e $k \geq 0$.
- Se $a > b^k$ então $T(n) = \Theta(n^{\log_b(a)})$
- Se $a = b^k$ então $T(n) = \Theta(n^k \log(n))$
- Se $a < b^k$ então $T(n) = \Theta(n^k)$

$$T(n) = nk \sum_{i=0}^{\log_b(n)} \left(\frac{a}{b^k}\right)^i$$

■ Exercícios:

Aplique o teorema mestre para resolver as recorrências a seguir, sempre que possível.

- $T(n) = 2T\left(\frac{n}{2}\right) + n$
- $T(n) = 5T\left(\frac{3n}{4}\right) + 7n^3$
- $T(n) = 6T\left(\frac{n}{8}\right) + 200\sqrt{n}$
- $T(n) = 7T(n - 1) + n^2$
- $T(n) = 8T\left(\frac{n}{2}\right) + n\log(n)$
- $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$

Perguntas?

`thiago.p.araujo@fgv.br`